Azure Interview Supplement — 3 Core Questions + Follow-Ups

Companion to: Azure-Interview-20-Questions.pdf
Topics: Azure Functions & Triggers · Service Bus / RabbitMQ · Application Insights
Project context: Champion Portal Azure deployment practice

Question A — Azure Functions & Its Triggers

Interviewer: Explain Azure Functions and its triggers. When would you use Functions instead of App Service?

Context to set:
Mention your hands-on: Consumption plan Function App, HTTP trigger intent, `AzureWebJobsStorage`, and .NET Isolated vs Node.js portal limitations.

Answer:

Azure Functions is **serverless, event-driven compute**. You run small code units without managing a full web server. On the **Consumption plan**, you pay mainly when code runs, and the app can **scale to zero** when idle.

Structure:

```
Function App          →  Host (runtime, settings, hosting plan)
   └── Function       →  Single unit of code with one primary trigger
         └── Bindings  →  Optional extra inputs/outputs
```

Common triggers:

Trigger	Fires when	Example
HTTP	API/webhook request	REST endpoint, webhook handler
Timer	CRON schedule	Nightly job, hourly sync
Queue	Storage queue message	Background processing
Blob	File upload/change	Image/CSV processing
Service Bus	Queue/topic message	Enterprise async workflows
Event Grid	Azure/custom event	React to resource changes
Cosmos DB	Document change	Update search index
Event Hub	Event stream	IoT / high-volume telemetry

Key terms:

Term	Meaning
Consumption plan	Serverless billing, scale to zero, possible cold start
Cold start	Delay on first request after idle

Function key	Secures HTTP functions (`?code=` or header)
AzureWebJobsStorage	Required storage connection for Functions runtime
Worker runtime	Language: `node`, `dotnet`, `python`

Hands-on from my deployment:

- Created `func-demo-http-ashish` on **Consumption (Windows)** with **Node.js**
- Learned **.NET 8 Isolated** requires VS Code/CLI — not in-portal create
- Fixed missing `AzureWebJobsStorage` by creating Storage account `stfuncdemoashish`

Functions vs App Service:

Use Functions	Use App Service
Webhooks, schedules, queue handlers	Full web app (React + API)
Short event-driven tasks	Long requests, middleware, SPA
Scale to zero acceptable	Always-on user-facing app

One-liner:

> *"Functions are event-driven and serverless; triggers define what starts them. I used Consumption plan, understand HTTP triggers and storage requirements, and choose App Service for Champion Portal's full-stack UI."*

Follow-Up Questions — Azure Functions

F1. What is a cold start and how do you reduce it?
Cold start is latency when Consumption plan wakes from zero instances. Reduce with Premium plan pre-warmed instances, Flex Consumption "Always Ready", or accept latency for low-traffic jobs.

F2. What is AzureWebJobsStorage and why is it mandatory?
Connection string to Storage account used by Functions host for scaling coordination, trigger leases, host locks, and internal state — not necessarily your business database.

F3. What are the HTTP authorization levels?
Anonymous (public), **Function** (function key required), **Admin** (master key). I use Function level for secured demos.

F4. Why couldn't you create a .NET function in the Portal?
.NET Isolated worker model requires deployment via Visual Studio, VS Code, or Azure Functions Core Tools. Portal in-editor creation targets Node.js, Python, PowerShell on supported plans.

F5. Give an example of using Functions with Champion Portal.
After activity submission, API writes to SQL and sends Service Bus message; Azure Function processes notification email or points calculation asynchronously.

Question B — Azure Service Bus / RabbitMQ

Interviewer: Compare Azure Service Bus and RabbitMQ. What happens when a Service Bus message fails?

Context to set:
Be honest: Service Bus was studied for interview prep; Champion Portal currently uses synchronous API → SQL. Show you understand messaging patterns and failure handling.

Answer:

Both are **message brokers** that decouple producers from consumers for **asynchronous, reliable** communication.

```
Tight coupling:     API ──direct call──► Service B
Loose coupling:     API ──message──► Queue/Topic ──► Service B (when ready)
```

Azure Service Bus (Azure-managed PaaS):

Feature	Purpose
Queues	Point-to-point; competing consumers
Topics + Subscriptions	Pub/sub; one event, many handlers
Dead-letter queue (DLQ)	Stores messages that failed after retries
Sessions	Ordered processing for related messages
Duplicate detection	Prevent double processing

When a Service Bus message fails:

```
1. Transient failure     → automatic retries
2. Max retries exceeded  → message moved to Dead-letter queue (DLQ)
3. Investigate DLQ       → inspect payload, exception, poison message
4. Fix handler           → ensure idempotent processing
5. Reprocess or abandon  → resubmit corrected message or discard
```

RabbitMQ (open-source, self-hosted or managed):

Aspect	RabbitMQ	Azure Service Bus
Hosting	Self-managed / CloudAMQP	Azure-managed
Model	Exchanges, queues, bindings	Queues, topics, subscriptions
Azure integration	Manual	Native (Functions, Logic Apps)
Best for	Multi-cloud, existing RabbitMQ teams	Azure-first architecture

Champion Portal example (future architecture):

Step	Action
1	User submits activity → API saves to Azure SQL
2	API sends message to Service Bus queue
3	Azure Function (Service Bus trigger) sends notification or updates analytics
4	API responds quickly; heavy work is async with retries

One-liner:

> *"Service Bus is Azure-managed messaging with queues, topics, and DLQ for failed messages. RabbitMQ is open-source and self-hosted. I'd use Service Bus in Azure for decoupled async work and DLQ-based failure investigation."*

Follow-Up Questions — Service Bus / RabbitMQ

F1. Queue vs Topic in Service Bus?
Queue = one consumer group processes each message once. Topic = pub/sub; multiple subscriptions each get a copy for different handlers.

F2. What is a poison message?
Message that always fails processing (bad format, bug). After retries it goes to DLQ so it doesn't block the queue indefinitely.

F3. What is idempotency and why does it matter?
Handler must safely process the same message twice without duplicate side effects — retries and at-least-once delivery can cause duplicates.

F4. Service Bus vs Storage Queue?
Storage Queue is simpler and cheaper for basic scenarios. Service Bus adds topics, sessions, transactions, DLQ, and enterprise features.

F5. When would you NOT use messaging?
Simple CRUD with low latency requirements and no need for decoupling — direct API → SQL is fine (like my current Champion Portal demo).

Question C — Have You Used Azure Application Insights?

Interviewer: Have you used Azure Application Insights? What did you monitor and how did you use it?

Context to set:
This is a strong yes from hands-on work. Be specific about blades, metrics, and a real troubleshooting flow.

Answer:

Yes. I enabled Application Insights on `championportal-AI` (Champion Portal App Service) during Azure deployment.

Setup steps I performed:

1. App Service → Application Insights → **Enable**
2. Created `championportal-Insights` in Central India
3. Linked to Log Analytics workspace (auto-created)
4. Restarted app; verified `APPLICATIONINSIGHTS_CONNECTION_STRING` in settings

What I monitored:

Blade	What I saw / used it for
Overview	Failed request rate (~~0%), avg response time (~~50 ms)
Live Metrics	Real-time requests while testing login and dashboard
Search	Individual traces: `POST /api/auth/login`, page loads
Performance	Slowest operations and average duration

Failures	Exceptions when configuration was wrong

Production debugging workflow:

```
1. User reports error on live site
2. Application Insights → Failures → exception + stack trace
3. Search → drill into failed request (URL, status, duration)
4. Log stream → immediate console output on App Service
5. Fix config or code → redeploy → confirm recovery in Overview
```

What Insights tracks for ASP.NET Core:

- HTTP requests (URL, status code, duration)
- Dependencies (e.g. SQL Database calls)
- Exceptions with stack traces
- Availability and performance trends

Optional test I can describe:
Broke JWT key temporarily → login failed → saw errors in **Failures** → restored key → verified success in Search.

Production improvements I'd add:

- Alerts on failure rate and response time
- Availability tests pinging URL every 5 minutes
- KQL dashboards for ops team

Sample KQL:

```
requests
| where timestamp > ago(1h) and success == false
| project timestamp, name, url, resultCode, duration
```

One-liner:

> "*Yes — I enabled Application Insights on my App Service, used Live Metrics and Search to trace API calls, and Failures to debug errors. For production I'd add alerts and availability tests.*"

Follow-Up Questions — Application Insights

F1. Difference between Application Insights and Log stream?
Log stream = real-time console logs, immediate debugging. Application Insights = historical APM with request tracing, aggregation, alerts, and dashboards.

F2. What is a dependency in Application Insights?
Outbound call from your app — e.g. Azure SQL query, HTTP API call. Shows duration and failure rate per dependency.

F3. How would you find a slow API endpoint?
Performance blade → Operations sorted by duration. Or KQL: `requests | summarize avg(duration) by name | order by avg_duration desc`.

F4. How do you alert on production errors?
Azure Monitor alert rule on Application Insights metric — e.g. failed request count > threshold → email/Teams via action group.

F5. Can you use Application Insights with Azure Functions?
Yes — enabled during Function App creation; same telemetry model for executions, failures, and dependencies.

Mock Interview Drill — Rapid Fire (All 3 Topics)

#	Question	10-second cue
1	Name 4 Function triggers	HTTP, Timer, Queue, Blob
2	What plan scales to zero?	Consumption
3	Required Functions storage setting?	AzureWebJobsStorage
4	Failed Service Bus message goes where?	Dead-letter queue (DLQ)
5	Service Bus vs RabbitMQ hosting?	Azure-managed vs self-hosted
6	What is a poison message?	Always fails; goes to DLQ
7	Application Insights blade for real-time?	Live Metrics
8	Blade for individual HTTP traces?	Search / Transaction search
9	Blade for exceptions?	Failures
10	Insights connection in App Service?	APPLICATIONINSIGHTS_CONNECTION_STRING

60-Second Combined Answer (If Asked All Three)

> *"I deployed Champion Portal on App Service with Application Insights — I monitored requests, failures, and response times using Live Metrics and Search. I also explored Azure Functions on Consumption plan with HTTP triggers and learned that AzureWebJobsStorage is required for the runtime. For async decoupling I'd use Azure Service Bus with dead-letter queues for failed messages; RabbitMQ is the self-hosted alternative when not Azure-native. App Service hosts my full web app; Functions handle event-driven background work."*

Supplement to Champion Portal Azure interview preparation.